

SIGNALFORGE

Multi-Strategy Quantitative Trading Intelligence Platform

You are going to act as my **senior quantitative developer, trading-system architect, data engineer, backend engineer, frontend engineer, and technical mentor.**

I want you to help me build this project from the ground up.

The project is called:

SignalForge

Multi-Strategy Quantitative Trading Intelligence Platform

1. CORE IDEA

SignalForge is NOT primarily an ML stock-price prediction system.

The core of the project is a **trading-strategy evaluation and selection engine.**

A user enters a stock symbol such as:

RELIANCE.NS

SignalForge retrieves historical market data and evaluates a library of approximately 10–12 rule-based trading strategies.

Examples:

- Bollinger Band Mean Reversion
- RSI Mean Reversion
- MACD Crossover
- SMA Crossover
- EMA Crossover
- Donchian Breakout
- N-Day Rate-of-Change Momentum
- Stochastic Oscillator
- ADX Trend Following
- ATR/Volatility Breakout
- Moving Average Pullback
- other strategies you determine are technically appropriate

Each strategy is backtested independently.

SignalForge then determines:

1. Which strategies performed best historically.
 2. Which strategies are most robust.
 3. Which strategies perform best under the current market regime.
 4. What signal the strongest strategy/strategy ensemble currently produces.
 5. Whether current company news supports or contradicts that signal.
 6. Whether similar historical news/events produced similar price reactions.
 7. The final BUY / HOLD / SELL research signal.
 8. The confidence and reasoning behind that signal.
-

2. IMPORTANT TERMINOLOGY

When I say "models", I mean **trading strategies**, not machine-learning models.

Do NOT turn the core project into:

XGBoost + LSTM + Random Forest stock prediction.

Machine learning may optionally be introduced later as an advanced extension, but the core system must be based on **rule-based trading strategies and rigorous backtesting**.

The central problem is:

"Which trading strategy is most suitable for this stock under its historical and current market conditions?"

3. EXAMPLE STRATEGIES

Build a modular strategy framework.

Every strategy should follow a common interface.

For example:

```
generate_signal(data)
calculate_position(data)
backtest(data)
get_parameters()
```

The exact architecture is your decision.

STRATEGY 1 — BOLLINGER BAND MEAN REVERSION

Concept:

Bollinger Bands create a normal price range around a moving average based on recent volatility.

Potential rule:

BUY: Price falls below the lower Bollinger Band.

SELL: Price returns toward the middle band or reaches an appropriate exit condition.

Parameters should be configurable:

- lookback period
- standard deviation multiplier
- entry threshold
- exit threshold

Do not assume these parameters are optimal.

STRATEGY 2 — RSI MEAN REVERSION

Concept:

Use RSI to identify potentially oversold/overbought conditions.

Example:

BUY: $RSI < 30$

SELL: $RSI > 70$

Make thresholds configurable.

STRATEGY 3 — MACD CROSSOVER

Concept:

Use the relationship between fast and slow moving averages.

BUY: MACD crosses above signal line.

SELL: MACD crosses below signal line.

Parameters should be configurable.

STRATEGY 4 — SMA CROSSOVER

Example:

BUY: Short-term SMA crosses above long-term SMA.

SELL: Short-term SMA crosses below long-term SMA.

Example parameters:

50-day / 200-day

But test multiple parameter configurations.

STRATEGY 5 — EMA CROSSOVER

Similar to SMA crossover but uses exponential moving averages.

Example:

20 EMA crossing 50 EMA.

Again, parameters must be configurable.

STRATEGY 6 — DONCHIAN BREAKOUT

Concept:

Trade breakouts from recent highs/lows.

Example:

BUY: Today's closing price breaks above the highest price/close of the previous N days.

SELL: Price breaks below the lower channel or an appropriate exit condition.

Test different N values.

STRATEGY 7 — N-DAY RATE OF CHANGE MOMENTUM

Concept:

Instead of looking only at yesterday's movement, evaluate momentum over N days.

Formula:

$ROC = (Current\ Price / Price\ N\ days\ ago) - 1$

Possible signal:

BUY: ROC exceeds a configurable positive threshold.

SELL: ROC falls below a configurable threshold.

Test multiple N values.

STRATEGY 8 — STOCHASTIC OSCILLATOR

Use overbought/oversold behavior.

Example:

BUY: Stochastic < 20

SELL: Stochastic > 80

Parameters must be configurable.

STRATEGY 9 — ADX TREND FOLLOWING

Use ADX to determine whether a strong trend exists.

Example:

ADX > configurable threshold

combined with directional movement.

This strategy should help distinguish trending markets from sideways markets.

STRATEGY 10 — ATR / VOLATILITY BREAKOUT

Use ATR to determine abnormal volatility expansion.

Combine volatility expansion with a price breakout.

The exact implementation should be justified.

STRATEGY 11 — MOVING AVERAGE PULLBACK

Identify an established trend and enter when price temporarily pulls back toward a moving average.

Example concept:

Bullish trend: Price > long-term moving average

Pullback: Price approaches short-term moving average

Potential BUY: Bullish confirmation after pullback.

STRATEGY 12 — ANOTHER ROBUST STRATEGY

Recommend one additional strategy based on:

- technical diversity
- usefulness
- ability to backtest
- difference from existing strategies

Do not add a strategy merely to increase the number.

4. STRATEGY DIVERSITY

The strategy library should contain different philosophies.

We should have:

Mean Reversion

Examples:

- Bollinger Bands
- RSI
- Stochastic

Trend Following

Examples:

- SMA crossover
- EMA crossover
- MACD
- ADX

Momentum

Examples:

- ROC
- momentum strategies

Breakout

Examples:

- Donchian
- ATR breakout

This is important because the system should discover that different strategies work better in different market environments.

5. HISTORICAL DATA

Initially support Indian equities/NSE symbols.

The user should be able to search for a stock.

Historical data should preferably cover:

2015 → present.

Use a reliable market-data provider/API.

The exact provider should be selected based on:

- availability
- reliability
- historical depth
- API limits
- ease of deployment

Do not hard-code API credentials.

Use environment variables.

6. DATA PROCESSING

Create a proper data pipeline.

Retrieve:

- Date
- Open
- High
- Low
- Close
- Adjusted Close if available
- Volume

Clean:

- missing values
- duplicate timestamps
- invalid rows
- corporate-action issues where applicable

Make sure all indicators use only information available at that point in time.

Avoid look-ahead bias.

7. TECHNICAL INDICATOR ENGINE

Create a reusable indicator layer.

Potential indicators:

- SMA
- EMA
- RSI
- MACD
- Bollinger Bands
- ATR
- ADX
- Stochastic
- ROC
- volatility
- momentum
- rolling returns

Strategies should consume these indicators instead of duplicating calculations.

8. PARAMETER OPTIMIZATION

This is an important feature.

Strategies should not be evaluated using only one arbitrary parameter configuration.

For example:

Bollinger Bands:

```
lookback:
10
20
30
50

standard deviation:
1.5
2.0
2.5
3.0
```

MACD:

```
8/21/5
12/26/9
20/50/9
```

Donchian:

```
N = 10
N = 20
N = 50
N = 100
```

ROC:

```
N = 5
N = 10
N = 20
N = 60
```

The system can evaluate parameter combinations on the calibration/training period.

BUT:

Do not overfit.

Use walk-forward validation and keep a genuinely unseen final test period.

9. TRAINING / VALIDATION / BLIND TESTING

These are rule-based strategies, so use the terms:

- calibration
- validation
- out-of-sample testing

rather than machine-learning "training" where appropriate.

Suggested structure:

```
2015–2021
Calibration period

2022–2024
Validation / strategy selection

2025–present
Final blind out-of-sample test
```

The exact dates should be configurable.

The final blind test period must not influence strategy selection or parameter optimization.

This is critical.

10. WALK-FORWARD BACKTESTING

Implement walk-forward evaluation.

Example concept:

```
Train/calibrate:
2015–2018

Validate:
2019

Then:

2015–2019
→ 2020 validation

Then:
```

2015–2020
→ 2021 validation

Continue appropriately.

The purpose is to simulate how the strategy would actually have behaved without knowing future data.

11. BACKTESTING ENGINE

Every strategy must be backtestable.

The engine should simulate:

- entry
- exit
- position
- cash
- portfolio value
- transaction costs
- slippage where appropriate

The system must support:

- long-only trading initially
- configurable transaction costs
- configurable initial capital

Do not implement short selling unless it is genuinely useful and technically justified.

12. STRATEGY PERFORMANCE METRICS

Do NOT rank strategies using total profit alone.

Calculate:

Returns

- total return
- CAGR

Risk

- volatility
- maximum drawdown

Risk-adjusted

- Sharpe ratio
- Sortino ratio

Trading statistics

- win rate
- profit factor
- number of trades
- average trade return
- average winning trade
- average losing trade

Robustness

- performance by year
- performance by market regime
- performance across validation windows

13. STRATEGY SCORING ENGINE

Create a normalized strategy score.

The score should consider:

- risk-adjusted return
- CAGR
- maximum drawdown
- Sharpe
- consistency
- trading quality
- robustness

The weights should be configurable.

For example:

Risk-adjusted return	30%
CAGR	20%
Drawdown	15%
Sharpe	15%
Win rate	10%
Stability	10%

These are initial design choices, NOT scientifically proven optimal weights.

Allow them to be changed.

14. STRATEGY RANKING

Display something like:

Strategy	Score
Donchian Breakout	87.2
MACD Crossover	82.6
EMA Crossover	78.4
ROC Momentum	76.9
ADX Trend	74.2
Bollinger Reversion	69.8
RSI Reversion	67.3
...	

The user should be able to inspect why each strategy received its score.

15. CURRENT MARKET REGIME

Create a market-regime detection component.

Possible regimes:

- Strong Bull Trend
- Weak Bull Trend
- Sideways
- Weak Bear Trend
- Strong Bear Trend
- High Volatility
- Low Volatility

Use objective indicators such as:

- trend
- moving averages
- ADX
- volatility
- momentum

Do not create arbitrary classifications without documenting the rules.

16. REGIME-AWARE STRATEGY SELECTION

This is one of the most important features.

A strategy that performs well overall may not be the best strategy under the current market regime.

Therefore calculate:

Overall Strategy Score

AND

Strategy Score within current market regime

Example:

Donchian Breakout

Overall Score: 87

Bull Market Score: 94

Sideways Score: 51

Bear Market Score: 64

Then:

Bollinger Mean Reversion

Overall Score: 70

Bull Market Score: 58

Sideways Score: 92

Bear Market Score: 74

If the current regime is sideways, SignalForge should consider Bollinger more strongly.

The final strategy-selection logic must be transparent.

17. CURRENT STRATEGY SIGNAL

After selecting the most suitable strategy, run that strategy on the most recent market data.

Example:

Selected Strategy:

Donchian Breakout

Current Signal:

BUY

Reason:

Price broke above the 50-day high.

Signal Strength:

82/100

18. STRATEGY CONSENSUS

Optionally calculate a broader strategy consensus.

Example:

10 strategies evaluated

BUY: 6

HOLD: 3

SELL: 1

Display:

Strategy Consensus:

Moderately Bullish

This can be used as supporting evidence.

Do NOT let consensus automatically override the best strategy.

19. NEWS INTELLIGENCE ENGINE

After obtaining the strategy signal, retrieve recent company-related news.

The system should perform:

1. News retrieval
 2. Deduplication
 3. Company relevance detection
 4. Sentiment analysis
 5. Event classification
 6. Recency weighting
 7. Importance scoring
-

20. NEWS SIGNAL

Each important article should receive:

- sentiment
- relevance
- importance
- event category
- timestamp

Example:

News Sentiment:
Negative

News Impact:
High

Relevance:
94%

News Score:
28/100

21. HISTORICAL NEWS SIMILARITY

For important current news:

1. Find similar historical company news/events.
2. Use semantic similarity where appropriate.
3. Determine what happened to the stock after those events.
4. Calculate average subsequent returns.
5. Calculate the number of comparable historical events.
6. Calculate confidence based partly on sample size and similarity.

Example:

Current event:
Major acquisition

Similar historical events:
17

Average 1-day return:
+1.2%

Average 5-day return:
+3.6%

Average 20-day return:
+5.1%

Historical Event Signal:
BULLISH

If there are too few similar events, clearly state:

"Insufficient historical evidence."

Never fabricate results.

22. FINAL SIGNAL ENGINE

Combine:

1. Selected strategy signal
2. Strategy performance
3. Current market regime
4. Strategy consensus
5. News sentiment
6. Historical news-event reaction

Example conceptual weighting:

Strategy / Backtest Evidence	40%
Current Strategy Signal	20%
News Intelligence	20%
Historical Similar Events	10%
Strategy Consensus	10%

These weights should be configurable.

Do not claim they are objectively optimal.

The final engine should produce:

Final Score: 78/100

SIGNAL:
BUY

Confidence:
76%

23. CONFLICTING SIGNALS

The system must handle disagreement.

Example:

Selected Strategy:
BUY

Strategy Consensus:
BUY

News:
STRONGLY NEGATIVE

Historical Similar News:
NEGATIVE

Market Regime:
SIDEWAYS

Instead of blindly saying BUY:

Final Signal:
HOLD

Confidence:
57%

Explain why.

24. EXPLANATION ENGINE

Every final signal should explain itself.

Example:

WHY?

- Donchian Breakout is currently the highest-ranked strategy.

- The stock has entered a bullish breakout.
- The strategy performs strongly in the current market regime.
- 6/10 strategies currently indicate BUY.
- However, recent company news is negative.
- Similar historical news events produced mixed returns.

Conclusion:

Moderately bullish, but uncertainty remains high.

25. BACKTEST COMPARISON

Create a dedicated backtesting page.

Allow users to compare:

Bollinger
MACD
Donchian
RSI
EMA
ROC
...

on the same stock and date range.

Display:

- equity curves
- returns
- drawdowns
- Sharpe
- Sortino
- CAGR
- win rate
- number of trades

Also compare each strategy against:

BUY & HOLD

This is essential.

26. STRATEGY LAB

Create a dedicated page called:

Strategy Lab

The user should be able to:

- select a stock
- select strategies
- select parameters
- choose date range
- run backtests
- compare results

Display a strategy leaderboard.

Example:

Strategy	Return	Sharpe	Drawdown	Win Rate
Donchian	42%	1.31	-14%	54%
MACD	36%	1.18	-17%	52%
EMA	31%	1.12	-19%	51%
...				

27. MAIN DASHBOARD

The dashboard should allow:

Search stock:
[RELIANCE.NS]

Then show:

Stock Overview

- current price
- daily change
- trend
- volatility
- market regime

Strategy Analysis

- best strategy
- strategy score
- strategy signal
- strategy consensus

News

- recent news
- sentiment
- impact

Historical Events

- similar news
- historical reaction

Final Signal

BUY

Confidence:
76%

Strategy:
Donchian Breakout

Why?

Provide a concise explanation.

28. TECHNICAL CHARTS

Include:

- candlestick/price chart
- moving averages
- Bollinger Bands
- RSI
- MACD
- volume
- strategy entry/exit markers
- buy/sell signals

Allow the user to select which indicators are displayed.

29. TECHNOLOGY STACK

Choose a realistic stack for a 4–5 week project.

Recommended starting architecture:

Frontend: React + TypeScript

Styling: Tailwind CSS

Charts: Plotly / Recharts / TradingView-compatible solution where appropriate

Backend: Python + FastAPI

Data/Backtesting: Python + Pandas + NumPy

Technical Indicators: A reliable Python technical-analysis library where appropriate, or implement indicators ourselves when useful for transparency.

Database: PostgreSQL

Caching: Use Redis only if actually necessary.

Deployment: Frontend on Vercel or equivalent.

Backend on a suitable cloud service.

Use Docker where useful.

Do NOT over-engineer the infrastructure.

30. DATABASE

Design a sensible database containing entities such as:

Stocks

HistoricalPrices

Strategies

StrategyParameters

BacktestRuns

StrategyMetrics

StrategySignals

NewsArticles

NewsAnalysis

HistoricalEvents

FinalSignals

Do not create unnecessary tables.

31. API

Create clean backend endpoints such as:

```
GET /stocks/search  
  
GET /stocks/{symbol}  
  
GET /stocks/{symbol}/history  
  
GET /stocks/{symbol}/strategies  
  
POST /backtest  
  
GET /backtest/{id}  
  
GET /stocks/{symbol}/best-strategy  
  
GET /stocks/{symbol}/signal  
  
GET /stocks/{symbol}/news  
  
GET /stocks/{symbol}/historical-events  
  
POST /analysis
```

Use proper validation and error handling.

32. PERFORMANCE

Do not run every backtest from scratch every time the user searches for a stock.

Use:

- caching
- persisted backtest results
- reusable historical data
- reusable indicator calculations

Allow the user to request a fresh analysis when necessary.

33. NO DATA LEAKAGE

This is one of the most important requirements.

Never allow future prices to influence:

- indicators
- parameter selection
- strategy ranking
- market regime classification
- news-event analysis

All signals must use information available at the time of the simulated trade.

Be particularly careful about:

- using closing prices to execute at the same close
- future news timestamps
- rolling calculations
- parameter optimization
- selecting strategies using the final test set

Document all assumptions.

34. TRANSACTION COSTS

Backtesting must include configurable:

- brokerage
- fees
- slippage

Use reasonable defaults but allow the user to modify them.

Show whether results are:

Before costs

or

After costs.

Prefer reporting after-cost results as the realistic result.

35. REALISTIC TRADING ASSUMPTIONS

Document:

- execution timing
- price used for entry
- price used for exit
- transaction costs
- slippage
- position sizing
- whether fractional shares are allowed
- whether overnight positions are allowed

Do not hide these assumptions.

36. POSITION SIZING

Initially keep the system simple.

Support:

- fixed percentage allocation
- equal capital allocation

Do not immediately implement complicated portfolio optimization.

The focus is strategy selection.

37. TESTING

Create tests for:

- indicators
- strategies
- signals
- backtesting
- transaction costs
- strategy scoring
- market regime classification

- news scoring

Most importantly test for:

LOOK-AHEAD BIAS

Create tests ensuring future information cannot influence past signals.

38. PROJECT STRUCTURE

Use a clean architecture.

Something similar to:

```
signalforge/  
  
  frontend/  
  
  backend/  
  
  strategy_engine/  
    strategies/  
    indicators/  
    backtesting/  
    optimization/  
    scoring/  
    regime/  
  
  news_engine/  
    ingestion/  
    sentiment/  
    events/  
    similarity/  
  
  data/  
  
  database/  
  
  tests/  
  
  scripts/  
  
  docs/
```

README.md

You may modify this structure if you have a better architecture.

39. DEVELOPMENT ROADMAP

I have approximately 4–5 weeks.

Create a realistic roadmap.

WEEK 1 — DATA + FOUNDATION

Build:

- repository
 - frontend
 - backend
 - database
 - stock search
 - historical market data
 - data cleaning
 - indicator engine
 - basic charts
-

WEEK 2 — STRATEGY ENGINE

Implement:

- Bollinger
- RSI
- MACD
- SMA crossover
- EMA crossover
- Donchian
- ROC
- Stochastic
- ADX
- ATR breakout

Build:

- strategy interface
- signal generation
- parameter system
- backtesting engine

- transaction costs
-

WEEK 3 — STRATEGY EVALUATION

Build:

- parameter optimization
- walk-forward validation
- strategy ranking
- performance metrics
- regime analysis
- Strategy Lab
- strategy comparison
- buy-and-hold comparison

This should be the technical core of the project.

WEEK 4 — NEWS INTELLIGENCE + FINAL SIGNAL

Build:

- news API
 - sentiment
 - event classification
 - historical news database
 - semantic similarity
 - historical event reaction
 - final scoring engine
 - confidence
 - explanation engine
-

WEEK 5 — POLISH + DEPLOYMENT

Build:

- polished dashboard
- advanced charts
- testing
- performance optimization
- caching
- Docker
- deployment
- documentation
- screenshots
- demo mode
- final README

40. WHAT MAKES THIS PROJECT DIFFERENT

Do not allow the project to become:

"Buy when $RSI < 30$."

The core research question is:

Which strategy works best for which stock, under which market conditions, and does that strategy continue to work on previously unseen data?

The platform should answer that question quantitatively.

41. FUTURE ML EXTENSION

Machine learning can later be added as an OPTIONAL advanced layer.

For example:

A meta-model could learn:

```
Stock features
+
Market regime
+
Strategy performance
+
Volatility
+
News sentiment
+
Strategy signals
↓
Probability that each strategy will outperform
```

However:

Do not build this initially.

First build the rule-based strategy-selection system correctly.

The ML meta-model can be a future extension after the core system is stable.

42. FINANCIAL DISCLAIMER

The application is a research and educational tool.

Display:

"SignalForge provides analytical signals for research and educational purposes. It does not constitute financial advice and does not guarantee future market performance."

Never claim that a BUY signal guarantees profit.

Always expose uncertainty.

43. DEVELOPMENT RULES

Do not immediately dump the entire project into one response.

First:

1. Understand the project.
2. Identify problems with the design.
3. Propose the architecture.
4. Recommend the strategy library.
5. Recommend data sources.
6. Recommend the technology stack.
7. Design the database.
8. Design the API.
9. Create the development roadmap.

Then wait for my approval before implementation.

Once implementation begins:

- work incrementally
- inspect existing code before changing it
- don't overwrite working code unnecessarily
- explain important decisions
- test each major component
- fix errors
- maintain the architecture throughout the project

Write code suitable for a serious portfolio project.

44. FIRST RESPONSE

For your FIRST response to this prompt:

DO NOT START CODING.

Instead provide:

1. Your understanding of SignalForge.
2. Improvements you recommend to my concept.
3. The 10-12 strategies you recommend.
4. Why each strategy is useful.
5. The strategy parameter-optimization approach.
6. The backtesting methodology.
7. The walk-forward validation methodology.
8. The strategy scoring methodology.
9. The market-regime methodology.
10. The news architecture.
11. The historical-news similarity architecture.
12. The final BUY/HOLD/SELL decision architecture.
13. Recommended technology stack.
14. Database architecture.
15. API architecture.
16. Frontend architecture.
17. 4-5 week development plan.
18. Biggest technical risks.
19. How we will prevent overfitting and look-ahead bias.

Do not begin implementation until the architecture is clearly defined.